

UCS503 - SOFTWARE ENGINEERING

Student Academic Manager

A Centralized Web-Based System for Academic Organization and AI-Assisted Learning

Submitted by:

1024030621 Ria Bansal

1024030619 Avneet Kaur Bhatia

1024030604 Ishaan Garg

Group: 3C44 BE Third Year, CSE

Submitted to:

Dr. Raghav B. Venkataramaiyer

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	4
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	5
2.1.2	Technical Stack	6
2.2	Implementation Details	7
2.2.1	Module 1: Core Functionality	8
2.2.2	Module 2: AI-Assisted Learning and Academic Planning . . .	8
3	Results and Evaluation	9
3.1	Testing Methodology	9
3.2	Performance Metrics	10
3.3	Analysis	11
4	Conclusion	12
4.1	Summary of Work	12
4.2	Key Findings	12
4.3	Future Work and Recommendations	12
4.4	Final Remarks	13
	Bibliography	14

Introduction

1.1 Background and Context

University students manage a large amount of academic information, including course syllabus, study materials, examinations, assignments, revision schedules, and performance records. In many cases, these resources are distributed across different platforms, folders, applications, and personal notes. This makes it difficult for students to maintain an organized study workflow and monitor their academic progress effectively.

The Student Academic Manager is a full-stack web application developed specifically to provide students with a centralized platform for managing their academic activities. The system combines academic organization, study-material management, examination preparation, performance tracking, and AI-assisted learning into a single application.

The application allows students to create and manage subjects, organize their syllabus into units, upload study materials, track the completion status of materials, and maintain their examination datesheet. It also provides AI-based features such as generation of university-style mock questions, evaluation of written answers, and AI-assisted study planning.

The system uses a client-server architecture in which the React frontend communicates with a Node.js and Express.js backend. MySQL is used for persistent storage of user accounts, subjects, units, study materials, mock questions, attempts, evaluations, examinations, and study-planner tasks. AI functionality is provided through the Google Gemini API.

By combining these features into one platform, the project aims to reduce fragmentation in academic management and provide students with a structured and intelligent study environment.

1.1.1 Problem Statement

Students often use multiple disconnected tools to manage their academic activities. Study materials may be stored in different locations, examination dates may be maintained separately, and performance tracking may require manual effort. In addition, students may have difficulty revising lengthy study materials and obtaining meaningful feedback on written answers.

The specific problem addressed by this project is the lack of a centralized academic management system that combines study-material organization, syllabus management, examination planning, performance tracking, and AI-assisted learning in a

single platform.

The proposed system addresses this problem by providing a web-based Student Academic Manager with the following capabilities:

- Centralized management of subjects and syllabus units. Uploading and organization of PDF, PPT, and PPTX study materials.
- Tracking of study-material completion status.
- AI-generated theoretical mock questions.
- AI-based evaluation of students' written answers.
- Examination datesheet and countdown management.
- Manual and AI-assisted study planning.
- Performance analytics and score visualization.
- Secure student authentication using institutional email addresses

1.2 Project Objectives

1. **Objective 1:** To develop a centralized academic management platform that enables students to manage subjects, syllabus units, study materials, examinations, and study tasks from a single web application.
2. **Objective 2:** To implement an AI-assisted learning system generate university-style theoretical questions, and evaluate student answers with scores, feedback, missing points, and model answers.
3. **Objective 3:** To develop an academic performance and planning module that allows students to maintain examination schedules, create study tasks, monitor completion, and visualize academic performance.
4. **Objective 4:** To implement secure user authentication and data management using institutional email verification, password hashing, JWT-based authentication, and a relational MySQL database
5. **Objective 5:** To provide a user-friendly and responsive web interface through which students can access their academic information and learning tools efficiently

Methodology and Design

2.1 System Architecture

The Student Academic Manager follows a three-layer client-server architecture consisting of the presentation layer, application layer, and data layer.

The presentation layer is implemented using React and provides the user interface through which students interact with the system. The application layer is implemented using Node.js and Express.js and is responsible for authentication, business logic, API handling, file processing, database communication, and AI integration. The data layer uses MySQL for persistent storage of academic and user information.

The major flow of the system is:

Student → React Frontend → REST API → Express Backend → MySQL Database

For AI-based features, the backend communicates with the Google Gemini API. Uploaded academic documents are processed by the backend, where text is extracted before being used for learning functionality.

2.1.1 High-Level Design

The system consists of the following major components:

1. Authentication Module

The authentication module handles student registration and login. Registration is restricted to institutional email addresses. Passwords are securely stored using hashing, while JWT tokens are used to maintain authenticated sessions.

2. Subject and Syllabus Management

Students can create, update, and delete subjects. Each subject can contain multiple syllabus units, allowing academic content to be organized according to the course structure.

3. Study Material Management

Students can upload PDF, PPT, and PPTX files and associate them with a particular subject and unit. The system stores information about the uploaded files and maintains their study status as *Not Started*, *In Progress*, or *Completed*.

4. Mock Practice and AI Evaluation

The system can generate theoretical questions based on a student's subject and study content. Students can submit written answers, after which the AI evaluates the response and provides a score, strengths, missing points, incorrect points, improvement suggestions, and a model answer.

5. Examination and Study Planner

The datesheet module allows students to maintain examination information including subject, date, time, location, and notes. The planner allows students to create study tasks and can also generate study plans using AI assistance.

6. Performance Analytics

The performance module collects examination and mock-practice information and presents academic performance using charts and comparative analysis.

2.1.2 Technical Stack

Framework/Language:

- JavaScript
- React 18
- Node.js
- Express.js
- HTML5
- CSS
- Tailwind CSS

Database:

- MySQL
- MySQL2 Node.js driver

AI Technology:

- Google Gemini API
- Gemini 3.6 Flash

Authentication and Security:

- JSON Web Tokens (JWT)
- bcryptjs
- Parameterized SQL queries

Development Tools:

- Visual Studio Code
- Git/GitHub
- MySQL Workbench
- Node Package Manager (npm)
- Vite

Libraries and Supporting Technologies:

- React Router
- Axios
- Recharts
- Lucide React
- Multer
- pdf-parse
- officeparser

React Router is used for frontend navigation, Axios is used for communication with the backend APIs, Recharts is used for performance visualization, and Multer is used for handling file uploads.

2.2 Implementation Details

The application was implemented using a modular full-stack architecture. The frontend is divided into reusable components, pages, contexts, services, and utilities, while the backend follows a controller-route architecture.

The backend contains separate controllers and routes for authentication, subjects, units, materials, summaries, mock practice, attempts, examinations, planning, performance, dashboard, and profile management.

The MySQL database uses relational tables with primary keys and foreign-key relationships to maintain consistency between users and their academic data.

2.2.1 Module 1: Core Functionality

The core functionality of the system consists of authentication, subject management, syllabus organization, and study-material management.

The authentication module provides registration and login functionality. Students are required to use University email address. Passwords are hashed before storage in the database, and JWT tokens are used for authenticated requests.

After authentication, students can create subjects and assign subject codes and descriptions. Each subject can contain multiple units, enabling students to represent their course syllabus in a structured manner.

The study-material module allows students to upload PDF, PPT, and PPTX files. The backend uses Multer for file handling and stores uploaded files in user-specific directories. Metadata such as filename, file type, file size, subject, unit, and study status is stored in the MySQL database.

The system also extracts text from supported documents using pdf-parse and officeparser. The extracted text is subsequently available for AI-assisted processing

2.2.2 Module 2: AI-Assisted Learning and Academic Planning

The second major module focuses on intelligent learning assistance and academic planning.

The Mock Practice module uses Gemini to generate theoretical questions according to the student's academic context. Questions can be categorized according to difficulty and question type.

The AI Evaluation module evaluates the student's written response. The evaluation includes a numerical score and multiple qualitative components such as conceptual understanding, accuracy, completeness, structure, examples, strengths, missing points, incorrect points, improvement suggestions, and a model answer.

The Study Planner module supports both manually created tasks and AI-generated study plans. Tasks contain information such as subject, unit, date, estimated duration, priority, and completion status.

The Performance module provides visual analytics of academic performance. This allows students to identify performance trends and compare their results across subjects and different types of assessments.

Results and Evaluation

3.1 Testing Methodology

The Student Academic Manager was tested at different levels to verify the correctness and integration of its major components.

Unit Testing

Individual backend controllers, API routes, authentication operations, database queries, and frontend components were tested independently. CRUD operations for subjects, units, examinations, and planner tasks were checked using different input conditions.

Authentication functionality was tested using valid and invalid credentials, while input validation was tested using incomplete and incorrect information.

Integration Testing

Integration testing was performed to verify communication between the React frontend, Express backend, and MySQL database.

The following workflows were tested:

- User registration and login.
- Subject creation and retrieval.
- Unit creation under a subject.
- Study-material upload.
- Retrieval of uploaded materials.
- Study-status updates.
- Mock-question generation.
- Answer submission and AI evaluation.
- Examination creation and retrieval.
- Planner task creation and completion.

- Performance-data retrieval.

Performance Testing

The application was tested during normal local development conditions to verify that API requests, database operations, file uploads, and frontend navigation function correctly.

Particular attention was given to database connectivity, API response handling, file-processing operations, and AI API requests because these operations directly affect the responsiveness of the application.

User Testing

The major workflows were evaluated from the perspective of a university student. The usability of navigation, subject organization, study-material management, mock practice, examination planning, and performance visualization was considered during testing.

3.2 Performance Metrics

Since this project is primarily a functional academic management application rather than a high-throughput production system, evaluation was focused mainly on functional correctness, feature completeness, integration, and usability:

Metric	Target	Achieved
User Authentication	Secure registration and login	Implemented
Subject Management	Complete CRUD operations	Implemented
Unit Management	Complete CRUD operations	Implemented
Study Material Management	PDF/PPT/PPTX upload and organization	Implemented
Study Status Tracking	Three study states	Implemented
Mock Question Generation	AI-generated questions	Implemented
AI Answer Evaluation	Score and detailed feedback	Implemented
Examination Management	Datesheet and countdown	Implemented
Study Planner	Manual and AI-assisted planning	Implemented
Performance Analytics	Visual performance analysis	Implemented

Table 3.1: System Requirements Achievement Summary

3.3 Analysis

The project successfully meets its primary objective of developing a centralized academic management platform. Students can organize subjects and syllabus units, upload study materials, maintain examination schedules, and manage study tasks from one application.

The AI integration extends the functionality beyond traditional academic management systems. The mock-practice module allows students to generate questions and receive automated evaluation of their written responses.

The relational database design provides structured storage for users and their academic information. Foreign-key relationships ensure that academic records remain associated with the appropriate users, subjects, and units.

One of the major challenges during implementation is the integration of multiple independent technologies. The project requires communication between React, Express, MySQL, file-processing libraries, and the Gemini API. Another challenge is handling uploaded documents and extracting usable text before sending the content for AI processing.

These challenges were addressed through modular backend controllers, separate API routes, middleware for authentication and file handling, parameterized SQL queries, and a dedicated Gemini service.

Conclusion

4.1 Summary of Work

The Student Academic Manager was developed as a full-stack web application to provide University students with a centralized academic management and learning platform.

The system provides authentication, subject and syllabus management, study-material organization, mock-question generation, AI-based answer evaluation, examination scheduling, study planning, and performance analytics.

The project integrates a React-based frontend with a Node.js and Express.js backend and a MySQL relational database. Google Gemini is integrated through the backend to provide AI-based academic assistance.

The implemented system fulfills the major objectives defined at the beginning of the project by combining academic organization and intelligent learning assistance into a single platform..

4.2 Key Findings

- **Finding 1:** A centralized academic platform can combine multiple student activities such as syllabus management, study-material organization, examination planning, and performance tracking into a single workflow
- **Finding 2:** AI integration can enhance traditional study-management systems by automatically practice questions, study plans, and feedback on written answers
- **Finding 3:** A relational database is effective for maintaining structured relationships between students, subjects, units, study materials, examinations, study tasks, questions, and evaluation records.
- **Finding 4:** Separating the frontend, backend, database, and AI service into independent components makes the application easier to maintain and extend

4.3 Future Work and Recommendations

The following improvements can be considered for future versions of the Student Academic Manager:

1. **Potential enhancements to the system:** Add notifications and reminders for upcoming examinations, incomplete study tasks, and pending revision activities.
2. **Alternative AI approaches:** Additional AI models could be integrated to generate summaries, questions, and evaluations and improve reliability.
3. **Scalability improvements:** The application could be deployed using cloud infrastructure with managed databases, object storage, caching, and load balancing to support a larger number of users.
4. **Integration with other systems:** The platform could be integrated with university academic portals, calendars, learning-management systems, or institutional course information systems.
5. **Real-world deployment:** Future versions could implement production-grade cloud deployment, HTTPS, automated backups, stronger monitoring, centralized logging, rate limiting, and enhanced security controls.
6. **Advanced analytics:** Machine-learning models could be introduced to identify weak subjects, predict academic performance, and recommend personalized revision schedules.
7. **Mobile support:** A dedicated mobile application or Progressive Web App could be developed to allow students to access their study plans and academic information conveniently from mobile devices.
8. **Collaborative learning:** Future versions could support sharing of study resources, collaborative notes, peer discussions, and group study planning.

4.4 Final Remarks

The Student Academic Manager demonstrates how modern web technologies and artificial intelligence can be combined to address practical academic-management challenges faced by university students.

The project provided practical experience in full-stack development, relational database design, REST API development, authentication, file processing, frontend development, and AI API integration. It also demonstrated the importance of designing modular systems in which individual components can be developed, tested, and extended independently.

The developed platform provides a foundation for a more comprehensive academic assistant that can evolve from a personal study-management application into a scalable intelligent learning platform for university students.

Bibliography

1. React Documentation. *React – A JavaScript Library for Building User Interfaces*. Meta Platforms, Inc.
2. Node.js Documentation. *Node.js Documentation*. OpenJS Foundation.
3. Express.js Documentation. *Express – Node.js Web Application Framework*.
4. MySQL Documentation. *MySQL 8.4 Reference Manual*. Oracle Corporation.
5. Google. *Gemini API Documentation*. Google AI for Developers.
6. Vite Documentation. *Vite – Next Generation Frontend Tooling*.
7. Tailwind CSS Documentation. *Tailwind CSS Documentation*.